

Shopping Cart System

Architecture Overview

Take-home assignment — Full Stack Engineer position, משרד הביטחון, מערכות המידע, אגף התקשוב ומערכות המידע

1. Overview

shopping-cart-system is a monorepo of three independently deployable services implementing a two-screen shopping flow: client (a React single-page app), server-catalog (a .NET 10 API backing the product catalog, screen 1), and server-orders (a NestJS API backing order submission and history, screen 2). All three share one git repository and one root docker-compose.yml/.env pair for local orchestration.

Each service goes a bit beyond the bare assignment — a rate limiter and structured logging in server-catalog, an Elasticsearch mapping wired directly into the running service in server-orders, cart persistence and a styles/component split in client, among other additions covered in this document. None of it is required by the brief; it reflects what a system meant to run past a demo would actually need, and every addition below is also explained in that service's own README.

Technology stack

Layer	Technology
client	React 19, Redux Toolkit + RTK Query, Vite, TypeScript, MUI, react-router, i18next (en/he, RTL/LTR)
server-catalog (screen 1)	.NET 10, ASP.NET Core, EF Core, SQL Server
server-orders (screen 2)	NestJS, Elasticsearch
Infrastructure	Docker Compose — SQL Server + Elasticsearch always on; all three services under an optional "full" profile

2. System Architecture

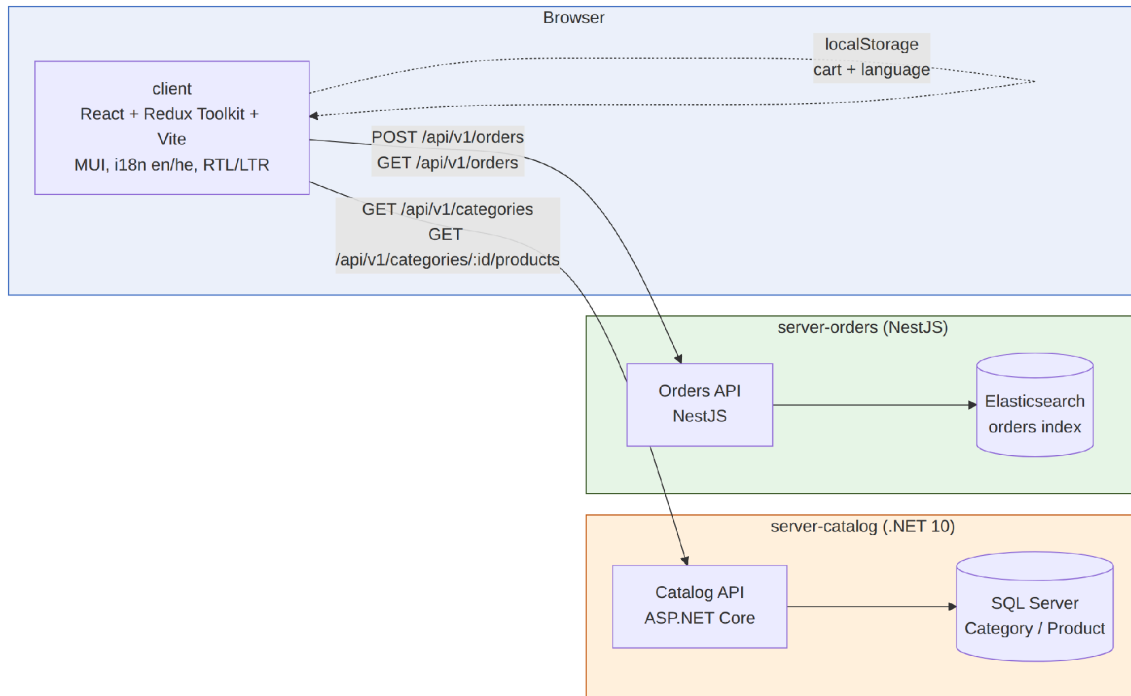


Figure 1. Components and the requests between them.

The two backend services are fully independent — server-orders never queries server-catalog's database. An order stores a denormalized snapshot of the product name, category and unit price the shopper saw at the time of purchase, rather than a live reference into the catalog, since the two services do not share a datastore. client talks to both APIs directly over HTTP and keeps the shopping cart and the language preference in the browser's localStorage, so a page reload loses neither.

3. Request Flows

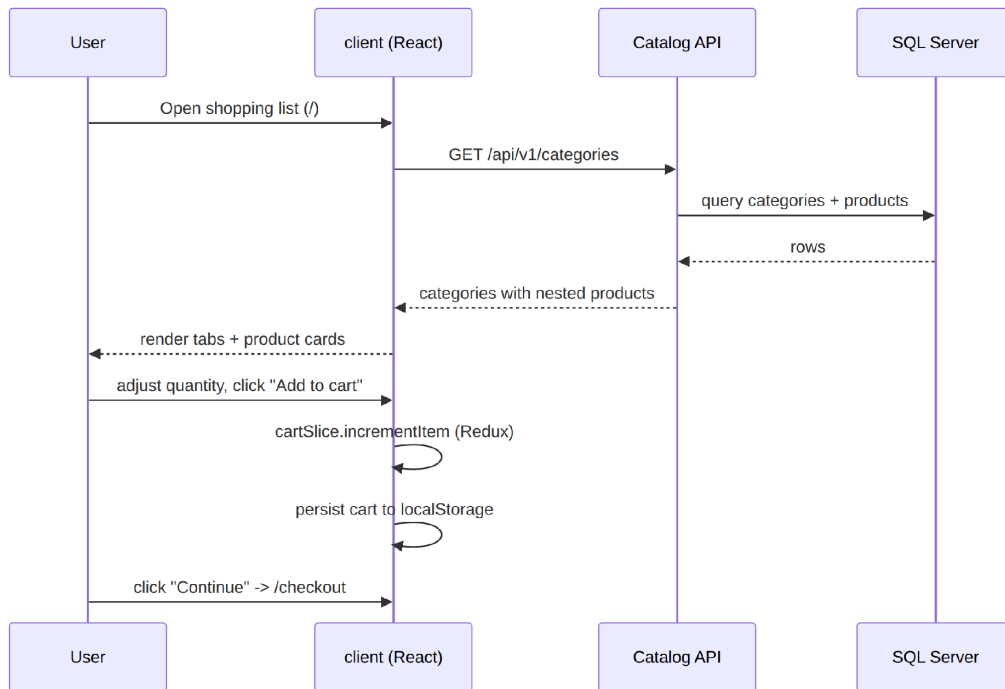


Figure 2. Screen 1 — shopping list (/).

Categories and their products load in a single request; the per-card quantity stepper doubles as the "add to cart" action, updating Redux state and persisting it to localStorage on every change.

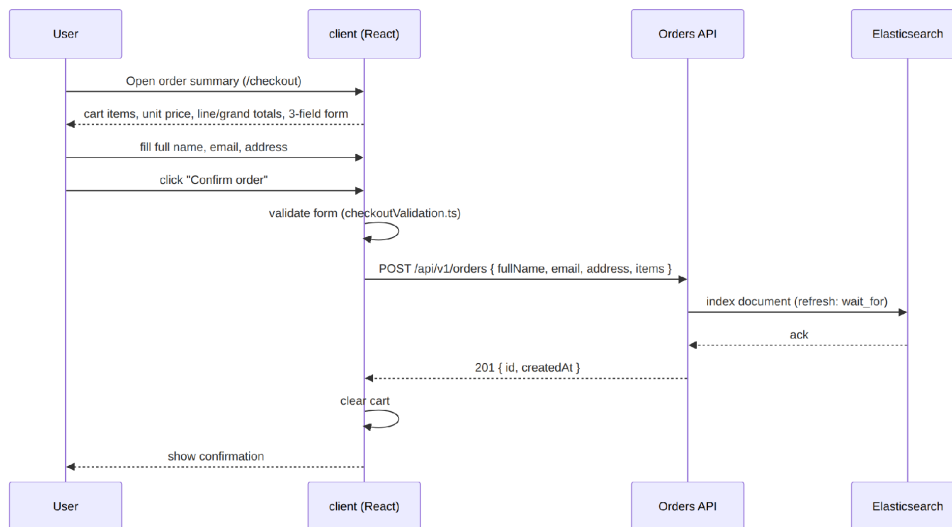


Figure 3. Screen 2 — order summary (/checkout).

The cart snapshot (unit price and unit, captured when each item was added) drives the per-line and grand totals shown here; the 3-field form (full name, email, address) is validated client-side before the order is posted. server-orders indexes the order with refresh: 'wait_for' so it is immediately visible in the order history, and the client clears its cart only after a successful 201 response.

4. Data Model

Catalog (SQL Server, via EF Core)

Entity	Fields
Category	Id (PK), NameEn, NameHe
Product	Id (PK), NameEn, NameHe, CategoryId (FK), ImageUrl (external source link), ImagePath (local, served path), UnitPrice (decimal), Unit (Kilogram Piece Liter)

One category has many products. Demo data — 3 categories, 18 products — is seeded from an embedded JSON resource shared by both the SQL-backed and in-memory repository implementations.

Orders (Elasticsearch mapping)

Field	Type
id	keyword
fullName	text + keyword sub-field
email	keyword
address	text + keyword sub-field
createdAt	date
items	nested — productId (integer), productName (text + keyword), categoryName (keyword), quantity (integer)

The mapping the assignment explicitly asks for lives in server-orders/mapping.json and is what the service loads to create the index on first start — not a copy that can drift. items is mapped nested rather than object so per-item queries stay correct; the mapping root sets dynamic: "strict" (rejects unmapped fields, matching the DTO's own validation) and a single shard with zero replicas, matched to the single-node deployment used for this assignment.

5. API Contracts

Method & path	Service	Description
GET /api/v1/categories	Catalog	Full catalog (all categories + products), unpaginated
GET /api/v1/categories/{id}/products?cursor&limit	Catalog	Cursor-paginated products with meta: { limit, total }
GET /health	Catalog	DB connectivity health check
POST /api/v1/orders	Orders	Create an order from a denormalized item snapshot
GET /api/v1/orders?cursor&limit	Orders	search_after-paginated order history with meta
GET /health	Orders	Elasticsearch connectivity health check

Both services share one error envelope — { status, message, errors[] } — and a common X-Correlation-Id convention through both services' request logs, so a request that crosses both APIs can be traced by the same header.

6. Key Design Decisions

Decision	Rationale
NestJS over Express	A layered architecture symmetric with the .NET side (modules, DI) rather than a minimally-assembled Express app.
Elasticsearch over MongoDB	Matches the assignment's stated preference, and exercises mapping/nested-type ownership — the mapping.json file the assignment explicitly asks for.
Denormalized order line items	The two services don't share a database; an order is a snapshot of what the shopper saw and agreed to, not a live link into the catalog.
fullName instead of first/last name	The brief specifies exactly three checkout fields; splitting the name would make four.
Cursor (keyset) pagination, both APIs	Page cost doesn't grow with depth and results don't shift under concurrent inserts, unlike offset-based paging.
In-process caching (IMemoryCache / RTK Query)	Sufficient for a read-mostly catalog at this scale; a shared cache (Redis) is deliberately deferred — see Future Improvements — until more than one instance needs to share state.
Rate limiting + a concurrency limiter, both APIs	A per-client sliding window plus a global in-flight cap, built entirely on each framework's own primitives rather than a new dependency.
Both services and datastores kept stateless	No server-side session; horizontal scaling needs no sticky routing today, at the cost of pushing shared caching to Future Improvements.
Composition-root DI in .NET; per-module DI in NestJS	Dependency wiring is visible in the file tree (a dedicated Catalog.DependencyInjection project) rather than buried in Program.cs / app.module.ts.
Cross-platform tooling throughout	No OS-specific scripts anywhere; setup helpers are single cross-platform Node scripts instead of parallel .ps1/.sh pairs.

7. Testing & Verification

Service	Framework	Coverage
server-catalog	xUnit.v3 + Moq	Category-to-DTO mapping logic, DI wiring across all three layers, in-memory repository data integrity and cursor pagination, cache-decorator behavior.
server-orders	Jest + Supertest	10 unit tests (service logic, cursor encode/decode) plus an 11-test e2e suite that boots the real Nest application against a mocked Elasticsearch client — no live cluster needed to run it.
client	Vitest + React Testing Library	33 tests — cart reducer logic and localStorage persistence, checkout form validation, checkout price/total calculations, and one interactive component test.

Tests target logic that can actually regress. Thin pass-through layers — a repository that is little more than an ORM query, or a controller that just delegates — are deliberately left untested, since a test there would mostly be re-testing the underlying framework rather than this project's own code.

8. Future Improvements

Framed as what would come next to take this from a take-home demo to something production-ready, not as gaps left unspoken.

System-wide

- **Authentication/authorization.** Add OAuth2/OIDC via an SSO provider in front of both APIs — today neither has an auth boundary, and order history carries PII (name, email, address).
- **Shared state in Redis.** Once running more than one instance of either service, move the in-process cache and rate-limit counters to a shared store so replicas don't diverge or under/over-count.
- **A message queue (Kafka or RabbitMQ).** Move work that needn't block the response — confirmation emails, cache invalidation, search reindexing — off the request path.
- **A reverse proxy (NGINX).** In front of both services once running multiple replicas, for load balancing and TLS termination.
- **A CI pipeline.** Build and test all three services on every push, plus Dependabot/Renovate for dependency freshness — server-orders' e2e suite is already CI-ready today.
- **Paginate the catalog listing itself.** GET /api/v1/categories returns everything at once; fine at 3 categories, not once the catalog actually grows.
- **Responsive images.** srcset/sizes on top of the lazy loading already implemented, once product images move to blob storage behind a CDN.

server-catalog

- **Real content management.** Replace the static JSON seed with an admin UI or product-management API.
- **Blob storage + CDN for product images.** Local files under wwwroot/images don't survive horizontal scaling.
- **Distributed tracing and metrics (OpenTelemetry).** Structured logs cover one instance; not a request that crosses services, or latency/error-rate over time.

server-orders

- **Filtering/search on order history.** By customer, date range, or status — cursor pagination covers walking the list, not finding one order.
- **A real multi-node Elasticsearch cluster.** With replica shards and TLS/auth, instead of the single-node demo instance.
- **Move off refresh: 'wait_for'.** Right for a demo's immediate read-after-write; won't scale under real write load.
- **A second integration suite against a real, disposable Elasticsearch container.** Layered on top of the existing mocked-client e2e suite, to validate Elasticsearch's actual query semantics.

client

- **Generate API clients from OpenAPI.** Instead of hand-written types, once the contracts stabilize.

- **End-to-end tests (Playwright).** Against real running instances of both APIs, covering the complete checkout lifecycle.